

Проект 13.17 — Автономный дрон без GPS

Навигация без GPS на основе компьютерного зрения и VINS-Mono.

- Реальный борт: NVIDIA Jetson Orin Nano (ARM64, JetPack 6 / L4T r36).
- Симуляция: ноутбук x86 + NVIDIA GPU (ArduPilot SITL + Gazebo Harmonic).

Словарь (с 2026-09-16, слово «боевой» не используем — ничего боевого тут нет):

бортовой — то, что крутится на Jetson (стек, конфиг, камера; противоположность сима); реальный борт / Orin — само железо; лётный — код/пути/нода, которые летят (противоположность стенда, реплея, болванки); штатный — дефолт/профиль по умолчанию; целевая архитектура — куда идём (VINS + NN1 вместо GPS).

Структура репозитория

```
docker/
  orin/      — БОРТОВОЙ стек (Jetson, реальная камера + полётник)
  sim/      — СИМУЛЯЦИЯ (x86 + NVIDIA: SITL + Gazebo + VINS)
src/
  camera/   — C++ CUDA камера-нода (camera_pkg) + tuner
  vins/     — VINS-MONO-ROS2 (конфиги) + python cam-ноды (fallback); calib/ —
              калибровка бортовой камеры/IMU Kalibr: мишени ПВХ 1×1 м / A1 / A4, версии, пайплайн
  sim/      — симуляционная обвязка (байеризатор, launch)
  control/  — пакет control_pkg: стабилизаторы, арбитр, гейты + ручки прогона
              (profiles/). Доки рядом: architecture.md (слои, порты),
              control.md (впрыск команд), gates.md (ГЕЙТЫ И СТРАХОВКИ:
              VINS Gate, мост VINS-EKF, guard'ы яруса и возврата — что
              проверяется и когда срабатывает; там же мост позы бута —
              единственный источник позы EKF до открытия моста, его
              цепочка отказа и замер, и почему гейт высоты у канала
              вида сверху выключен), speed.md (чем ограничена
              максимальная скорость), windspeed.md (стрелка ветра HUD),
              tune.md (настройка демпфера заново), low-rider.md (полёты
              ниже 1 м), yawbanklimit.md (потолок крена виража в LOITER)
  mission/  — пакет mission_pkg: ЛЁТНАЯ НОДА (bootstrap_arch2) и план полёта —
              шаги freefly/SoftLand/Rth, лесенка ярусов, строгая схема ручек
              BS_* (config.py), офлайн-тесты плана в test/; rth.md — ВОЗВРАТ
              ДОМОЙ: латч доверия, гейт зрелости моста, план возврата по треку
  lab/      — скрипты прогонов (freefly_lv.sh, capture_scene.sh, cmd/* зовут их)
              и ОФЛАЙН-СТЕНДЫ по bag: vins_sane_replay, ipm_alt_replay,
              rth_check, hud_video/ipm_video, joystick/ (реплей пульта,
              js_probe, joy_timeline). Таблица инструментов — src/lab/CLAUDE.md
  nav/      — пакет nav_pkg: нейросети навигации (NN1/NN2, пока болванки)
              + openhd_streamer (даунлинк в OpenHD с оверлеем детекций)
  orin_shutdown/ — Go-утилита graceful shutdown через MAVLink
distro/     — ИСТОЧНИК ПРАВДЫ конфигурации Jetson (etc/, home/andriy/, usr/,
              doc/, deploy.sh): правим локально → deploy.sh → проверка по ssh,
              на борту руками ничего не менять (distro/CLAUDE.md)
  doc/HW/hw.txt — ПАСПОРТ РЕАЛЬНОГО БОРТА: компоновка, вес, силовая, связь,
              датчики, цены — и раздел «Софт»: что где крутится на борту
              и в симуляции. Первое, что читать про железо
tools/mdtopdf/ — генератор CLAUDE.pdf (reportlab)
```

Бинарники в репе — пустые заглушки *__bin (реальные собираются на месте).

Навигация (два слоя)

Нейросеть №1 — якорная локализация (замена GPS, абсолютная точность):

YOLOv8 / SuperPoint+LightGlue находит известные ориентиры → Ray Tracing через intrinsics + барометр/IMU → абсолютная позиция → сбрасывает дрейф VINS-Mono.

Нейросеть №2 — топологическая карта (семантика): DINOv2 + AnyLoc + FAISS, сжимает сцену в дескриптор, сравнивает с базой облёта → управление «по смыслу».

Нейросеть №2 ведёт дрон, Нейросеть №1 периодически сбрасывает дрейф.

Реализация — пакет nav_pkg (src/nav/, ament_python). См. «OpenHD-оверлей».

Камера и C++ нода

src/camera/camera_node.cpp (camera_pkg) — лётная ROS2 нода:

- V4L2 захват (v4l2-ctl, формат BA10) → CUDA дебайер (cv::cuda::demosaicing, BayerGB2RGB) → публикует mono8 в /image_mono (вход VINS)
- публикует /image_color (bgr8, полный кадр, каждый кадр) — вход nav-стороны (нейросети + openhd_streamer). Цвет приводится к настоящему BGR (cv::cuda::cvtColor RGB→BGR); моно-путь VINS считается ДО конверсии (не изменён)
- ROS-параметры на лету: gain/r/g/b; device (путь к V4L2, default /dev/video0) — ключ к работе в симуляции (см. ниже)
- stream_openhd (default false) — встроенный энкодер OpenHD :5600.

По умолчанию ВЫКЛ: поток теперь собирает openhd_streamer (рисует рамки NN).

true — standalone-режим камеры без nav-стороны

- штампует кадр через get_clock()->now() → уважает use_sim_time

src/camera/tuner/cuda/main.cpp — автономный веб-тюнер (без ROS): :8080 GUI, :5000 mono MJPEG, :5001 color MJPEG. Запускается вручную.

Разные коды Байера в CUDA (BayerGB2RGB) и CPU (BayerGR2BGR) — норма: в OpenCV CUDA-модуле сдвиг в именовании паттернов. Оба обрабатывают один физический паттерн (GRBG сенсора).

OpenHD-оверлей и nav_pkg (вариант 2)

Видео для оператора (OpenHD) проходит через нейросети, которые рисуют рамки вокруг объектов. Нейросети работают РЕДКО (NN1 ~1 Гц, NN2 ~3 с) и НЕ должны гейтить fps видео. Поэтому энкодер вынесен в отдельную ноду, а нейросети шлют только геометрию/семантику (килобайты, не пиксели):

```

camera_node → /image_color ──┬─> nn1_anchor (1Гц) → /nn1/detections (Detection2DArray)
                             └─> nn2_scene (3с) → /nn2/scene (String, метка)
                             └─> openhd_streamer ← кэш последних детекций
                                           рисует оверлей на КАЖДОМ кадре → H.264 → OpenHD :5600

```

src/nav/ = пакет nav_pkg (ament_python). Раскладка по слоям (модули разнесены по подпапкам, имена нод/executables НЕ менялись):

- nav_pkg/openhd_streamer.py — общий стример (не NN1/NN2);
- nav_pkg/nn1/ — якорь: nn1_anchor, ray_tracer, либы anchor_matcher, geo;
- nav_pkg/nn2/ — топокарта: nn2_scene, relocalizer, либа scene_descriptor;
- tools/nn1/, tools/nn2_scene/ (подход 1: топограф/изометрия),

tools/nn2_route/ (подход 2: route-координаты $-□V$, концепт-ветка) — офлайн-скрипты.

• openhd_streamer — подписан на /image_color + /nn1/detections + /nn2/scene, кэширует последние детекции, рисует на каждом кадре (cv2.rectangle/putText): рамки NN — на полном кадре, затем ужимает до 640×360, debug-HUD — уже на ужатом (текст нативно, без размытия ресайзом), кодирует H.264 (GStreamer → udpsink :5600). Видео на полном fps независимо от инференса; рамки «залипают»

между обновлениями (для FPV норм). Debug-HUD (параметр hud, default true):

баннер гейта LOITER-на-VINS (зел/жёлт/крас) из /mission/status — ego публикует лётная нода bootstrap_arch2 (ТОТ ЖЕ гейт, что пускает LOITER — единый источник правды; логика control_pkg/application/hud.py, офлайн-тест test_hud_status.py); постоянный баннер статуса борта — машина состояний «EKF WARMUP» (жёлт) → «EKF READY - TAKEOFF OK» (зел) → «ARMED» (зел) → после дизарма снова READY/WARMUP (поле ekf= той же строки, критерий = WaitEkfPos — свежий local_position; в полёте ARMED, не ekf); строка «ALT baro X.Xm ekf X.Xm perc X.Xm» — высота ТРЕМЯ источниками (alt= rel_alt миссии, баро при BS_ALT_SRC=baro; zekf= z того же local_position глазами EKF3; palt= высота ПЕРЦЕПЦИИ — по ней и судит гейт земли IPM; протухшее честное --), расхождение больше $\max(0.2, 0.2 \cdot \text{baro})$ — жёлтым (порог относительный: на полёте 0.3 м прежние фиксированные 0.5 м не срабатывали); строка «IPM <код> NN%» — живость канала вида сверху (ipm=ipmf=, доля годных за 3 с; красный ALT GATE = осям запретили по высоте — так выглядели прогоны 183305/185921, где демпфер выдал 0 PWM); плюс

Гц/возраст /odometry (меряет сам), фичи трекера

(/feature | /feature_tracker/feature) — счётчик FEAT и ЗЕЛЁНЫЕ ТОЧКИ

самых фич на кадре (пиксельные u,v из каналов PointCloud [id,u,v,vx,vy];

параметр hud_features), armed /mavros/state (баннер статуса борта),

PWM-смещения демпферов /flow_dbg* (PWM = канал пульта/стика, не ШИМ моторов), дрейф /nn1/drift. Без лётной ноды

(голый стример на Orin) баннер гейта не рисуется, остальное живёт.

/mission/status пишется в bag (freely_lv.sh), joy_timeline показывает переходы «HUD: LOITER READY» / «HUD: ярус 1 VINSHOLD» в ленте событий; баннер и блок ярусов (3.2/3.6 в hud.md) — ЛЕСЕНКА SF-мастера (потолок SC, активный ярус, гейт каждого яруса с прогрессом; отдельной строки режима FCU над блоком нет с 2026-08-31 — дубль, имя режима ушло в хвост строки активного яруса), живость VINS — только строка ODO (разбор ab_noise 2026-08-30: «VINS WAIT» при живом VINS — ждал ярус, не VINS). Баннер LANDING ... — мягкая посадка по кнопке SA (land=sa= в статусе; шаг SoftLand плана freely, config.ff_land): в LOITER → штатный LAND на EKF-от-VINS со стеком пустым (в position-LAND стик = уставка скорости, демпфер там вредит), иначе — в т.ч. до инициализации VINS — снижение в ALT_HOLD под демпфером/VinsHold (стик = наклон) 0.15 м/с (LAND_SPEED 15 в ветке LAND), касание → газ в пол → дизарм. Второе нажатие SA до касания ОТМЕНЯЕТ посадку (с 2026-09-06, BS_FF_LAND_CANCEL, дефолт 1) в любой ветке и на любом ярусе: из LAND — keep (ALT_HOLD) сразу и ре-ассерт 5 с со стиками в центре, пока FCU не вышел из LAND (один async set_mode терялся — отмена в LOITER не срабатывала), goto freely, стек и опора от текущей точки, ярус перепринимают лесенка; после касания отмены нет. Скорость LAND (LAND_SPD_MS 0.15) = land_rate ветки ALT_HOLD — одинакова на всех ярусах. □□ На ярусе LOITER кнопка SA ОТКЛЮЧЕНА (BS_LAND_IN_LOITER, дефолт 0): отмена из LAND полётника кнопкой не сработала и после ре-ассерта — садить из LOITER руками либо CH6 вниз → ярус 0/1 → SA.

Мягкость = скорость касания замкнутым контуром vz FCU, не доля газа хOVERA (70–80 % хOVERA с 1 м = 2 м/с об землю). Гейт кнопки: $rel_alt \leq 5 \text{ м}$, $|v| \leq 1 \text{ м/с}$. Без ROS-тест: src/mission/test/test_freely_land.py.

В scene.mp4 HUD НЕТ по построению

(это чистый /image_color); полёт «глазами пилота» — scene_hud.mp4: пост-рендер из bag тем же кодом (nav_pkg/hud_renderer.py, без ROS) через src/lab/hud_video.py (freely_lv.sh делает сам, HUD_MP4=0 — выкл).

Третий ролик того же прогона — scene_ipm.mp4, «глазами КАНАЛА ВИДА СВЕРХУ»: полоса земли, нарисованная на кадре лётным ipm_px, + сам выпрямленный варп + лётные скорости /flow_dbg8|9 рядом с истинной Gazebo (src/lab/ipm_video.py, рисовалка ipm_panel.py общая с офлайн-стендом ipm_alt_replay.py; IPM_MP4=0 — выкл). Варп в bag не пишется — он пересчитывается настоящим FlowEstimator по лётному конфигу.

Каждое поле HUD по отдельности — источник, цвета, пороги, таймауты, чек-лист «что смотреть по фазам полёта»: src/nav/hud.md.

- nn1_anchor — Нейросеть №1 (якорная локализация). Инкремент 1: SuperPoint+ LightGlue (anchor_matcher.py) матчит /image_color против географической базы облёта (data/reference_db/) → bbox+id ориентирует в /nn1/detections.
- ray_tracer — Инкремент 2: засечка по ориентире (geo.py: луч через

intrinsics + углы MAVROS + баро → абсолютная позиция в ENU) → поправка-смещение к VINS (сброс дрейфа) → /nn1/anchor_pose, /nn1/corrected_odom, /nn1/drift.

Инкремент 3: публикует скорректированную позу в /mavros/vision_pose/pose (ArduPilot EK3 External Nav) — ray_tracer = единственный мост VINS → полётник.

Гейт здоровья моста (nn1/bridge_gate.py, с 2026-09-06): vision_pose не идёт и якорь заморожен при $|twist| > 12$ м/с, перерождении потока (дыра/скачок позы), шторме подтяжек якоря (≥ 3 за 5 с) и по вердикту лётной ноды /vins/sane; перерождение → якорь латчится ЗАНОВО. Повод — полёт 142811: разнёсшийся VINS через 687 подтяжек отравил ориентацию EKF3 (крен/тангаж 6–12°, курс 20–33°), DrHold унесло.

Якорь: расход только XY и грейс 10 с после (пере)латча (anchor_grace_sec) — иначе после закрытия моста убежавший EKF и подтяжка «к EKF» запирали друг друга (полёт 173102).

Состояние моста — /nn1/bridge → brg=/brw=/brl=/brc= в /mission/status.

С 2026-09-09 добавлены: гейт зрелости (/vins/bridge_ok от лётной ноды: пока VINS не доказал себя, позу в EKF не пускаем вовсе — 2-4 с мусора хватало, чтобы EKF уехал на 200 м) и сброс кадра на первом открытии (anchor_open_reset_sec: 2 с шлём сырой VINS, чтобы полётник пересел на свежую раму, а не усыновил накопленный за закрытие дрейф). Возврат домой и латч доверия — src/mission/rth.md; рамы координат (ENU / кадр EKF по компасу / кадр VINS по первому кадру), шов якоря и план перехода на курс VINS (EK3_SRC1_YAW=6) — src/nav/frames.md.

Кадры выравнивает FrameAnchor (frame_anchor.py, офлайн-тест src/nav/test/test_frame_anchor.py): Дуав + трансляция латчатся по паре поз EKF/VINS — мир монокуляра рождается с курсом первого кадра, и спавн с курсом -169° разносил LOITER (положительная ОС, прогоны 2026-08-24). Осталось: yaw-коррекция ПО ЗАПЕЧКЕ + FAISS-префильтр. На FCU нужен EK3_SRC1_POSXY=6.

Детали и допущения: src/nav/tools/nn1/nn1_anchor_howto.txt.

- nn2_scene — Нейросеть №2 (топологическая карта). Инкремент 1: DINOv2 сжимает кадр в глобальный дескриптор, FAISS (scene_descriptor.py) ищет ближайшее место в карте облёта (data/scene_map/: map.index + metadata.json) → метка в /nn2/scene (баннер) + поза места (GPS/ENU + кватернион) в /nn2/relocalization. Цель — релокализация VINS после потери трекинга; пока допущение «VINS на треке», позу просто отдаём. Карта собирается из bag: tools/nn2_scene/build_scene_map.py (мок — generate_mock_map.py). Метрический MLP-«топограф» поверх DINOv2 (MetricHead в scene_descriptor.py: изометрия L2 □ метры; -mlp в build/eval, метрика карты l2) — голова РЕАЛИЗОВАНА + тренер tools/nn2_scene/train_topograph.py (дистанц-регрессия + triplet на дельтах VINS). Осталось: прогнать обучение на реальных bag'ах + детектор потери VINS + применение позы через ray_tracer + FAISS-префильтр для NN1.

Детали: src/nav/tools/nn2_scene/nn2_scene_howto.txt.

- relocalizer — ПУСТАЯ нода-заглушка: принимает /nn2/relocalization, логирует. Сюда ляжет восстановление VINS (поправка отдаётся в ray_tracer).

- Запуск: `ros2 launch nav_pkg nav.launch.py use_sim_time:=true` (камеру/VINS не поднимает). В симуляции включается из `sim_nav.launch.py` (IncludeLaunch).

Зависимость `vision_msgs` (`Detection2DArray`) — добавлена в образ `nav` (в `ros-base` её нет). Вариант 2 выбран ради чистоты архитектуры; межнодовый `republish` полного `/image_color` — осознанная плата, потом можно оптимизировать. Для NN2 в образ `nav` добавлен `faiss-cpu`; DINOv2 тянется в рантайме через `torch.hub` (использует уже стоящий `torch`). На Orin NN2 пока не запускается.

Реальный борт (`docker/orin/`): камера больше не гонит OpenHD сама (`stream_openhd:=false`) — `openhdl_streamer` запускается рядом с камерой в `vins_service.sh/vins_service_m.sh` (PID4). На Orin поднимается ТОЛЬКО стример (не болванки NN1/NN2): отдаёт чистое видео, рамки появятся, когда настоящие нейросети начнут публиковать `/nn1/detections`, `/nn2/scene.src/nav` смонтирован в контейнер, в `Dockerfile` добавлен `vision-msgs`.

VINS-Mono

Апстрим: <https://github.com/dongbo19/VINS-MONO-ROS2> (ROS2 Humble, ARM64 + x86).

Наш форк: <https://github.com/linux100talion/VINS-MONO-ROS2>, ветка `1317_debug`.

Все патчи проекта (QoS, IMU skip, debug-вывод в `imu_factor.h`, защита от стояния на земле — `rejectWithF` без `findFundamentalMat` при нулевом сдвиге точек (на неподвижных кадрах OpenCV 4.10 бросает `rowRange-assert`, трекер молчал 7-48 с) и сброс `INITIAL` при `>300` кадрах в `all_image_frame` без ключевых) живут в ветке форка. Парный фикс на стороне борта: лётная нода шлёт `/restart` на фронте `armed` (`BS_VINS_RESTART_ARM`, дефолт 1) — иначе накопленное на земле окно инициализации после отрыва не решается за полёт ($O(N^3)$; разбор в памяти `vins-ground-stand-init`).

C++ исходники (`feature_tracker`, `vins_estimator`) НЕ вендорены в репо — клонируются вручную рядом с `config_pkg`:

```
# на Orin (в контейнере vins_project_13_7):
cd /root/vins_ws/src
git clone --branch 1317_debug https://github.com/linux100talion/VINS-MONO-ROS2
colcon build --packages-select feature_tracker vins_estimator

# в симуляции — форк клонируется НА ХОСТ (не в контейнер):
git clone --branch 1317_debug https://github.com/linux100talion/VINS-MONO-ROS2 /root/VINS-MONO-ROS2
# затем монтируется в nav через bind mount (docker-compose.yml):
# /root/VINS-MONO-ROS2:/root/sim_ws/src/vins_oss
# colcon build запускается автоматически через nav_up.sh
```

Конфиги: `src/vins/VINS-MONO-ROS2/config_pkg/config/`

- `dummy_13_7.yaml` — бортовой (реальный ArduCam)
 - `sim.yaml` — симуляция (см. ниже)
-

Бортовой стек — docker/orin/

База: dustynv/ros:humble-ros-base-l4t-r36.3.0 (NVIDIA L4T: ROS2 Humble + CUDA + OpenCV-c-CUDA даром). runtime: nvidia, network_mode: host, проброс /dev/video0. Контейнер vins_project_13_7. Сборка через colcon внутри контейнера (монтируются src/vins, src/camera, src/nav).

Systemd (в distro/etc/systemd/system/): mavros, vins/vins_m (запуск VINS; суффикс _m = ручной режим без ожидания арминга), auto-bag/auto-bag-m, orin-shutdown. Скрипты — distro/home/andriy/vins_ws/.

distro/ *РАЗМОРОЖЕН 2026-09-16 (ветка laptop_drone, этап деплоя начал):*
каталог — источник правды конфигурации Jetson, цикл «правка локально → deploy.sh → проверка по ssh», на борту руками ничего не менять — правила, доступ (ssh, sudo) и таблица «что на борту устарело» в distro/CLAUDE.md.
Код в контейнер по-прежнему едет из src/ (bind mount), не из distro/.
Известные несоответствия, разобрать ПО ХОДУ ДЕПЛОЯ: vins_ws/vins_service.sh запускают python cam_node.py вместо C++ camera_node и не поднимают openhd_streamer (актуальная версия скриптов — git show 634119d:distro/home/andriy/vins_service.sh);*
vins_ws/src/plus/cuda/camera_node.cpp — древний монолит с битой строкой x264enc (лётный код — src/camera/, едет в контейнер bind mount'ом);
заглушка usr/local/bin/orin_shutdown__bin удалена. Секреты (doc/ssh-keys/jetson, doc/wifi.txt) — в .gitignore, репо публичный.
Раскладка каталога и сравнение бортовой записи bag (auto-bag) с симуляционной (freely_lv → capture_scene) — distro/CLAUDE.md.

Симуляция — docker/sim/

Запуск всей системы на ноутбуке БЕЗ железа. Три контейнера:

| Сервис | База | GPU | Роль |

|---|---|---|---|

| simulator | osrf/ros:humble-desktop + Gazebo Harmonic | graphics+compute | Gazebo + ArduPilot SITL + ros_gz_bridge |

| mavlink_router | radarku/mavlink-router | — | раздача MAVLink по UDP |

| nav | nvidia/cuda 12.2 + ros-base | compute | VINS + камера-нода + нейросети + MAVROS |

Bind mounts и named volumes в compose

`docker-compose.yml` использует два типа постоянного хранения:

- Bind mounts — конкретный путь с хоста монтируется в контейнер (исходники, скрипты, конфиги).
- Named Docker volumes — управляются Docker, данные не привязаны к пути хоста

(`nav_colcon_build`, `nav_colcon_install` для артефактов сборки).

| Bind mount | Контейнер | Зачем |

|---|---|---|

| `./scripts:/scripts:ro` | simulator, nav | entrypoint-скрипты: редактируются на хосте, применяются сразу без пересборки образа |

| `./config/sitl-extra.parm:/root/sitl-extra.parm:ro` | simulator | параметры SITL (`SCHED_LOOP_RATE 100` и др.) — выживают при `fresh-start` |

| `./worlds:/root/worlds:rw` | simulator | SDF-миры и модели дрона |

| `./output:/root/output:rw` | simulator | логи → `make logs` читает с хоста |

| `../../src/nav:/root/sim_ws/src/nav` | nav | исходники `nav_pkg`: пишешь на хосте, `colcon build` в контейнере |

| `../../src/camera, src/sim, src/vins` | nav | аналогично — код на хосте |

| `/root/VINS-MONO-ROS2` (хост) | `/root/sim_ws/src/vins_oss` | nav | форк `linux100talion/VINS-MONO-ROS2`, ветка `1317_debug`; хранится вне репо |

| `./output:/root/sim_ws/output` | nav | логи nav-стороны → тот же каталог `docker/sim/output/` |

| Named volume | Контейнер | Зачем |

|---|---|---|

| `nav_colcon_build:/root/sim_ws/build` | nav | `colcon build`-артефакты: выживают при `fresh-start` |

| `nav_colcon_install:/root/sim_ws/install` | nav | `colcon install`-артефакты: выживают при `fresh-start` |

| `sitl_eeprom:/root/sitl_state` | simulator | `eeprom.bin` SITL (одноразовая accel-калибровка + параметры): SITL запускается из этого каталога, `eeprom` выживает при `fresh-start`, сносится `make clean`. Пересборка — `make sitl-cal` (см. `docker/sim/CLAUDE.md`) |

Что выживает при `restart-all` (`stop/start`) и что теряется при `fresh-start` (`down/up`):

- Выживает всё в `bind mounts` и `named volumes`: исходники, конфиги, артефакты `colcon`. `fresh-start` НЕ удаляет `named volumes` (`docker compose down` без `-v`).

- Теряется всё внутри контейнера вне `mounts/volumes`: установленные через `apt/pip` пакеты. Критичные (`ros-humble-image-transport`, `numpy<2`) должны быть в `Dockerfile`.

- `vins_oss` теперь `persistent`: монтируется через `bind mount` из `/root/VINS-MONO-ROS2` на хосте (форк, ветка `1317_debug`). При `fresh-start` переподключается автоматически, клонировать повторно не нужно. Патчи хранятся в форке, не в `nav_up.sh`.

- Принудительная пересборка `colcon`: `make nav-rebuild` (очищает `build/install` внутри контейнера → очищает `volume` → `colcon` пересобирает).

- Полный снос включая `volumes`: `make clean` (`docker compose down --volumes`).

Ключевые решения по симуляции

1. Gazebo Harmonic (не Fortress) — по concept.txt

(docker/sim/doc/tmp/concept.txt; плагины gz-sim-*).

Нештатная пара для Humble, ставится вручную из репозитория osrfoundation.

2. GPU = NVIDIA CUDA (везде).

3. Вариант А — OpenCV+CUDA из исходников в образе nav. Камера-нода работает в симуляции «как есть» (с CUDA-дебайером). База nav — devel CUDA-образ.

□□ apt'шный cv_bridge тянет системный OpenCV (4.5.4 без CUDA) → ABI-краш.

Поэтому cv_bridge собирается из исходников против нашего OpenCV (overlay /opt/overlay). CUDA_ARCH_BIN — build-arg под GPU ноута.

4. Камера через v4l2loopback (камера-нода НЕ переписывается):

`

Gazebo /camera/image_raw (RGB)

→ bayerizer.py (RGB → Bayer16)

→ write() → /dev/rawbayer (v4l2loopback, модуль ядра ХОСТА)

→ camera_node (device:=/dev/rawbayer) → CUDA дебайер → /image_mono

`

Меняется только параметр device. Байеризатор: src/sim/bayerizer.py (паттерн GRBG по умолчанию). v4l2loopback ставится на хосте (modprobe, симлинк /dev/rawbayer), пробрасывается в nav через devices:.

Особенности v4l2loopback (задокументированы при отладке на GCE):

- Поддерживает только BGR4 (32-bit BGRA). BA10/Y16 — EINVAL.

- VIDIOC_S_FMT требует буфер 208 байт (не 204): v4l2_format на x86_64 имеет 4 байта padding между type и union из-за выравнивания 8-байтного указателя v4l2_window.clips. Используем ctypes c_align = c_uint64 для автоматического учёта padding.

- ready_for_capture (доступность G_FMT для capture-стороны) **ставится только после первого write()**, не после S_FMT. Поэтому байеризатор открывает /dev/rawbayer и пишет один нулевой кадр в __init__().

- camera_node стартует с задержкой 4 с (TimerAction в sim_nav.launch.py), чтобы байеризатор успел активировать capture-сторону.

- camera_node читает sizeimage=3686400 (BGR4 1280×720×4), но проверяет frame_size_ >= wh2 и берёт только первые wh2 = 1843200 байт как cv_16UC1. Байеризатор пишет bayer16-данные + нули-паддинг до sizeimage.

- modprobe v4l2loopback ... width=1280 height=720 — подсказка по умолчанию.

5. isaac_ros удалён — драйвер камеры не поддерживает Argus.

6. Разрешение 1280×720 — под него захардкожена camera_node и посчитан sim.yaml. Камеру дрона в Gazebo держать 1280×720 (не 1920×1200).

7. `mavlink_router` — клиент к SITL (tcp:5760). В `concept.txt` был конфликт портов (два сервера на 5760) — исправлено.
8. `use_sim_time` всем нодам через `src/sim/sim_nav.launch.py`. Исключение — `ros_gz_bridge` (он источник `/clock`). MAVROS — отдельно (тонкий момент: часть штампов IMU от FCU).
9. Байеризатор вынесен из `sim_nav.launch.py` (запускается в `nav_up.sh` как отдельный `nohup`-процесс). Причина: если байеризатор падает внутри `launch`, `launch`-менеджер убивает весь дерево (`camera_node + VINS`). Вне `launch` — изолирован. `sim_nav.launch.py` содержит только `TimerAction(4с)` с `camera_node/VINS` и `IncludeLaunchDescription` для `nav_pkg`.
10. Entrypoint-скрипты (`auto-start` при старте контейнера). Скрипты `scripts/sim_up.sh` и `scripts/nav_up.sh` монтируются в контейнеры как `/scripts/:ro` и запускаются автоматически через `command`: в `compose`:

```
command: bash -c "bash /scripts/sim_up.sh; exec tail -f /dev/null"
```

Итерация разработки (без пересоздания контейнеров, `ephemeral state` жив):

```
make restart-all # docker compose stop → start
make logs # tail -f output/*.log
```

Полный сброс (пересоздать контейнеры):

```
make fresh-start # docker compose down → up
```

`nav` зависит от `simulator` (`depends_on:`), таймаут ожидания `/dev/rawbayer` увеличен до 60 с.

11. IMU `timesync` fix — SITL JSON-протокол возвращает `no_time_sync`, FCU использует `wall-time`; MAVROS с `use_sim_time:=true` считает смещение `ROS_now(Gazebo) - FCU_wall → offset` дрейфует → IMU timestamps перепецируют → VINS получает `imu message in disorder`. Решение двухслойное:
- MAVROS: `conn/timesync_mode:=NONE` — штампует все сообщения `rcLCPP::now()` (Gazebo sim-время), не синхронизирует часы с FCU.
 - `vins_estimator/estimator_node.cpp`, `imu_callback`: `skip`-патч — если `t <= last_imu_t`, делаем `return` (дубликат пропускается, не вставляется `1e-6`). Нужен потому что `/clock ~155 Гц < 250 Гц` IMU → несколько сообщений получают одинаковый timestamp. Вариант `1e-6` вызывал `numerical unstable in preintegration` (`dt²=1e-12` деградирует ковариационную матрицу) — заменён.

Оба патча зафиксированы в ветке 1317_debug форка linux100talion/VINS-MONO-ROS2 (bind-mounted из /root/VINS-MONO-ROS2); nav_up.sh патчи не применяет.

12. SITL параметры — docker/sim/config/sitl-extra.parm, bind mount /root/sitl-extra.parm:ro, подаётся в sim_vehicle.py через --add-param-file (scripts/sim_up.sh) — выживает при fresh-start.

Критичные:

```
SCHED_LOOP_RATE 100 # иначе "Main loop slow (249Hz < 400Hz)" → краш FCU
FS_GCS_ENABLE 0 # отключить failsafe GCS
ARMING_SKIPCHK -1 # выключить pre-arm чеки (4.8: бывш. ARMING_CHECK 0 —
# переименован, старая строка молча игнорировалась)
DISARM_DELAY 0
```

Мир — Military fortress

docker/sim/worlds/mili_fortress.sdf — карта из [engcang/gazebo_maps](https://github.com/engcang/gazebo_maps) (mili_tech), на которой в демо-видео доказанно работает VINS-Fusion+YOLO. Выбрана потому, что текстуры уже подходят для VINS (свой мир в Blender легко «промахнуться»). Ассеты вендорены в worlds/mili_tech/ (~27 МБ, CC; ATTRIBUTION.md).

Портирование Classic → Harmonic: добавлены обязательные system-плагины, <population> → явная расстановка, <road> убран, ode → DART, Ogre-материалы (grass_plane/digital_wall) → PBR, убрана битая ссылка model://home, фон-горы mt_background (heightmap на текстурах Ogre Classic) отключены в SDF.

□□ Базовая земля бесфактурна вблизи (gz-sim не тайлит текстуру по грани бокса: один grass_dry.png 512px растянут на 150 м, texel 29 см) — ближе ~3 м земля для зрения пуста: полнокадровый LK и IPM-канал слепнут у земли ПО ПОСТРОЕНИЮ МИРА (прогоны lv2 040737/041255). Для полётов у земли — вариант mili_fortress_fine.sdf (WORLD=/root/worlds/mili_fortress_fine.sdf, применяется на fresh-start): grass_plane_fine тайлит текстуру ГЕОМЕТРИЕЙ (сетка визуалов 15×15 м, grass_fine.jpg 4096px → texel 3.7 мм — предел нужного: варп IPM у пола 0.5 м сэмплирует 3.3 мм/px; многооктавная фактура 5мм-35см, йав тайлов рвёт периодичность); коллизия/высота/спавны не тронуты. С 2026-08-26 fine-мир — дефолт (строка WORLD в docker/sim/env.default, см. ниже); □□ прогоны на нём не сравнивать бит-в-бит со старыми сериями базового мира (сцена другая). База — закомментировать строку в .env.

Локальный профиль бокса docker/sim/.env (compose + дефолты

freely_lv.sh: LV=2, BS_SF_MASTER=1, WORLD) сеется из эталона
env.default (в git) автоматически при отсутствии — свежий клон летит
штатным профилем. Шпаргалка: docker/sim/env.md.

Дрон — worlds/iris_cam/

iris_with_ardupilot из ardupilot_gazebo (проверена под SITL: моторы, IMU, ArduPilotPlugin@9002) + камера пилота (параметры из concept.txt: поза 0.15 0 0.05, наклон 0.26, fov 1.5708; но 1280×720). Прикреплена к iris_with_standoffs::base_link, публикует gz-топик camera/image_raw.

sim.yaml (VINS для Gazebo)

Отличия от бортового: нулевая дисторсия (идеальный pinhole), интринсики $f_x=f_y=640$, $c_x=640$, $c_y=360$ (fov 90° @ 1280×720), пути /root/sim_ws/, экстринсики из позы камеры в SDF, заниженный шум IMU (нет вибраций рамы).

Полный пайплайн

```
gz sim (мир+дрон)
  → ArduPilotPlugin@9002 ↔ SITL → tcp:5760 → mavlink_router → MAVROS
  → ros_gz_bridge: /camera/image_raw + /clock
  → bayerizer.py (RGB→Bayer16, nohup, изолирован от VINS)
  → /dev/rawbayer (v4l2loopback, хост)
  → camera_node (device:=/dev/rawbayer) → /image_mono → VINS feature_tracker
                                         → /image_color → nav_pkg (nn1/nn2)
  → vins_estimator → /vins_estimator/odometry → ray_tracer → /mavros/vision_pose/pose
```

Запуск (после make host-setup и make build):

```
make up           # создать контейнеры; sim_up.sh + nav_up.sh запустятся автоматически
make logs         # смотреть output/*.log
make restart-all  # быстрый перезапуск без пересоздания (stop → start)
make fresh-start  # полный сброс (down → up, ephemeral state теряется)
```

□□ Дисциплина прогона (ОБЯЗАТЕЛЬНО)

Любой прогон симуляции — атомарная операция от перезапуска стека до записи bag.

Запускать пошагово вручную (особенно перезапускать в одиночку SITL/ноду внутри контейнера) ЗАПРЕЩЕНО — состояние стека рассинхронизируется (Gazebo жив, а FCU перезапущен → ложные диагнозы). Поведение нод воспроизводимо только из чистого старта всего стека.

Правила:

- Стек поднимать ТОЛЬКО целиком: make restart-all 2>&1 | tail -3 && make wait 2>&1.

Не делать `make sim`/ручной перезапуск отдельного процесса ради «прогона».

- Полный цикл атомарен: перезапуск → `make wait` → арминг/взлёт → скрипты из `src/lab/` → запись `bag`. Это одна неделимая последовательность.

- Без перезапуска можно ТОЛЬКО смотреть логи (`make logs`, `make status`, чтение `output/*.log`, `docker exec ... tail`). Никаких рестартов «по кусочкам».

- All-in-one: для повторяемости использовать единый скрипт поверх уже существующих (`src/lab/capture_scene.sh` — образец-секвенсор: сам делает `restart-all/fresh-start + wait` + последовательность лётных команд (`arm/takeoff/hover/land/disarm`) + запись `bag` + извлечение кадров). Новые сценарии оформлять так же — обёрткой над `make`/скриптами `src/lab/`, а не ручными командами в контейнере.

Облачный прогон — `gcp/`

GPU/CPU-прогоны `torch/CUDA/ROS2`, вынесенные с ноута/Jetson на GCE. Обёртки над `gcloud` для инстанса `dev-workspace-1317` + `bootstrap` нового бокса. Подробности и дисциплина (`build-box`, арх-привязка) — в `gcp/CLAUDE.md`.

Поднятие нового GCE-бокса — через `gcp/bootstrap_gce.sh` (не вручную):

запускается НА инстансе, ставит Node+Claude Code, клонирует репу и **подхватывает память из `gcp/memory/**` (едет в самой пене; `tarball/scp` — опц. `override`).

```
./gcp/bootstrap_gce.sh [REPO_DIR] [MEMORY_TARBALL]
```

Память (`gcp/memory/`) — снимок `~/claude/projects/-root-13-17/memory`; держать синхронной, чтобы новый бокс стартовал с тем же контекстом. Дальше — чек-лист `gcp/c3_gce_setup.txt` (зависимости + прогон пайплайна), план — `gcp/c3_TODO.txt`.

Репозиторий

- GitHub: <https://github.com/linux100talion/13.17>, ветка `main`
- Git user: Andriy Kutsevol andriykutsevol@gmail.com
- CLAUDE.pdf генерится: `~/venvs/pdfenv/bin/python3 tools/mdtopdf/claudepdf.py` (venv постоянный, переживает ребут; пересоздать: ``python3 -m venv ~/venvs/pdfenv && ~/venvs/pdfenv/bin/pip install -r tools/mdtopdf/requirements.txt``)